

Multi-armed Bandits

Resumen del capítulo 2

INTRODUCCIÓN TEÓRICO-PRÁCTICA AL APRENDIZAJE POR REFUERZO

Profesor: Matias Bilkis

Cuatrimestre: Segundo cuatrimestre 2026

Integrantes

Agustín Calo	390/23	Paz Lavenia	64/23
Rocco Gasparini	102/24	Silvano Picard	637/24
Maria Delfina Kiss	72/23	Ingrid von Foerster	150/23

Basado en *Reinforcement Learning: An Introduction*, de Richard S. Sutton y Andrew G. Barto.

Nota: no se incluye la parte de Gradient Bandits.

Idea general del capítulo

La idea central del **aprendizaje por refuerzo** (*Reinforcement Learning*, RL) es bastante simple: el agente no recibe la respuesta correcta de antemano. Prueba una acción y, a cambio, obtiene una **evaluación numérica de esa acción: la recompensa o *reward***.

Esto lo diferencia del aprendizaje supervisado:

El **feedback evaluativo** indica qué tan buena fue la acción realizada. Esto diferencia al aprendizaje por refuerzo del aprendizaje supervisado, que trabaja con **feedback instructivo**: ahí sí se indica cuál era la acción o respuesta correcta.

Como consecuencia, en RL el agente tiene que **probar acciones mediante prueba y error** para descubrir cuáles son mejores.

El problema de los *multi-armed bandits* simplifica esta idea eliminando estados y consecuencias futuras. Solamente hay que elegir acciones y observar recompensas. Así podemos enfocarnos en uno de los problemas centrales de RL:

Exploración frente a explotación

Un multi-armed bandit, dadas n acciones posibles:

$$a_1, a_2, \dots, a_n.$$

En cada instante t :

1. elegimos una acción A_t ;
2. recibimos una recompensa R_t ;
3. repetimos el proceso.

Cada acción tiene una distribución de recompensas propia, y el objetivo es **maximizar la recompensa total esperada a lo largo del tiempo**.

(El nombre viene de las máquinas tragamonedas: una máquina de una sola palanca es un *one-armed bandit*; aquí imaginamos una máquina con n palancas.)

Además, cada acción a tiene un valor verdadero $q(a)$, que representa su **recompensa esperada**:

$$q(a) = \mathbb{E}[R_t \mid A_t = a].$$

Si conociéramos todos los valores $q(a)$, resolver el problema sería trivial: elegiríamos siempre la acción con mayor valor. El problema es que **no conocemos esos valores**, por lo que tenemos que estimarlos.

Suponiendo que tenemos estimaciones de los valores, denotadas por $Q_t(a)$. La acción con mayor valor estimado se denomina *greedy action*.

Explotación

Consiste en elegir la acción que actualmente creemos que es mejor:

$$A_t = \arg \max_a Q_t(a).$$

Estamos aprovechando el conocimiento que ya tenemos. Su ventaja es que maximiza la recompensa esperada **inmediatamente**; su problema es que podríamos estar equivocados acerca de cuál es la mejor acción.

Exploración

Consiste en elegir una acción que actualmente no parece ser la mejor para obtener más información sobre ella. Nos permite descubrir acciones mejores, pero con el costo de posiblemente producir menos recompensa en el corto plazo.

Exploración frente a Explotación

No podemos explorar y explotar simultáneamente con una misma elección. Sacrificar recompensa ahora puede permitir obtener una recompensa mucho mayor en el futuro. La decisión óptima depende, entre otras cosas, de:

- las estimaciones actuales;
- la incertidumbre;
- la cantidad de pasos restantes;
- la estabilidad del problema.

Se buscan **métodos simples que consigan un buen equilibrio**.

Para **estimar cuánto vale cada acción**, la idea es hacer dos cosas:

1. estimar el valor de cada acción;
2. utilizar esas estimaciones para decidir qué acción elegir.

Se distingue entre $q(a)$, el valor **real** de la acción, y $Q_t(a)$, su valor **estimado** en el tiempo t .

Promediar las recompensas

La forma más directa de estimar el valor de una acción es promediar todas las recompensas que obtuvimos al elegirla. A esto se lo llama Simple-average method. Si elegimos a un total de $N_t(a)$ veces:

$$Q_t(a) = \frac{R_1 + R_2 + \dots + R_{N_t(a)}}{N_t(a)}.$$

Si la acción todavía nunca fue elegida, puede asignarse un valor inicial como

$$Q_1(a) = 0.$$

En un problema **estacionario**, si probamos la acción infinitas veces, entonces, por la ley de los grandes números,

$$Q_t(a) \longrightarrow q(a).$$

En palabras: la estimación del valor de la acción a se aproxima cada vez más a su valor verdadero a medida que aumenta la experiencia.

La estrategia más simple es elegir siempre la acción con mayor valor estimado. A esto se lo denomina Greedy Action Selection:

$$A_t = \arg \max_a Q_t(a).$$

Esta es una política *greedy*. El problema es que únicamente explota: si una acción tuvo mala suerte en sus primeras recompensas, podría parecer peor de lo que realmente es y no volver a ser elegida jamás.

Una solución muy sencilla consiste en introducir exploración aleatoria: ε -greedy

- con probabilidad $1 - \varepsilon$, elegimos la acción *greedy*;
- con probabilidad ε , elegimos una acción aleatoriamente.

Por ejemplo, con $\varepsilon = 0.1$, aproximadamente el 90 % de las decisiones son de explotación y el 10 % son de exploración.

Como existe una probabilidad positiva de elegir cualquier acción, a largo plazo todas las acciones son probadas repetidamente. Por lo tanto,

$$N_t(a) \longrightarrow \infty,$$

y las estimaciones terminan acercándose a sus valores verdaderos.

El libro compara estos algoritmos mediante un problema experimental con $n = 10$ acciones. Los valores verdaderos $q(a)$ se generan con una distribución normal:

$$\mu = 0, \quad \sigma^2 = 1,$$

y las recompensas también contienen ruido aleatorio. Los resultados se promedian sobre **2000 problemas diferentes durante 1000 pasos**. Se comparan:

- *greedy*: $\varepsilon = 0$;

- ε -greedy con $\varepsilon = 0.01$;
- ε -greedy con $\varepsilon = 0.1$.

El método *greedy* funciona ligeramente mejor al principio, pero después queda estancado porque frecuentemente encuentra una acción subóptima y deja de explorar. En el experimento encontró la acción óptima solamente en aproximadamente **un tercio de los problemas**.

En cambio, ε -greedy continúa explorando y eventualmente descubre mejores acciones. Con $\varepsilon = 0.1$ se descubre rápidamente la mejor acción, pero se sigue explorando bastante incluso después. Con $\varepsilon = 0.01$ se aprende más lentamente, pero a largo plazo se explota más la acción óptima. Incluso se podría hacer que

$$\varepsilon \longrightarrow 0$$

con el tiempo.

Cuanto mayor sea el ruido de las recompensas, más difícil es identificar la mejor acción y mayor valor tiene la exploración. También es fundamental en problemas **no estacionarios**, donde

$$q_t(a)$$

puede cambiar con el tiempo. La expresión se lee “ q sub t de a ” y significa el valor verdadero de la acción a en el instante t . Una acción que antes era mala puede convertirse posteriormente en la mejor. Por eso, el equilibrio entre exploración y explotación es una característica fundamental de RL.

Supongamos que queremos calcular

$$Q_n = \frac{R_1 + \dots + R_n}{n}.$$

La forma más directa de hacer esta cuenta sería guardar **todas las recompensas anteriores**. El problema es que cada vez necesitaríamos más memoria y más cómputo. Por suerte, podemos actualizar el promedio de a poco a poco. Si ya tenemos Q_k y recibimos R_k :

$$Q_{k+1} = Q_k + \frac{1}{k} (R_k - Q_k).$$

Así solamente necesitamos almacenar el valor actual Q_k y el contador k . Y de esta manera, podemos actualizar estimaciones sin guardar todo

Una regla que nos aparece

Esta fórmula es importante porque tiene una estructura que se vuelve a ver una y otra vez en aprendizaje por refuerzo:

$$\boxed{\text{Nueva estimación} = \text{Estimación anterior} + \text{Tamaño de paso} [\text{Objetivo} - \text{Estimación anterior}]}.$$

De forma abreviada:

$$\boxed{\text{Nueva estimación} = \text{Estimación anterior} + \alpha \times \text{Error}},$$

donde

$$\text{Error} = \text{Objetivo} - \text{Estimación anterior}.$$

La nueva estimación se mueve parcialmente hacia un **objetivo** (*target*). Esta estructura es una de las ideas más importantes del capítulo porque reaparece constantemente en algoritmos posteriores.

Hasta ahora veníamos suponiendo que $q(a)$ era constante. Pero en muchos entornos

$$q_t(a) \neq q_{t+1}(a).$$

En esos casos, promediar **toda la historia** puede jugar en contra: las recompensas de hace mucho tiempo quizás ya no representen lo que está pasando ahora.

Para dar mayor importancia a las observaciones recientes se utiliza

$$Q_{k+1} = Q_k + \alpha (R_k - Q_k), \quad 0 < \alpha \leq 1,$$

con α constante.

Al expandir la fórmula se obtiene

$$Q_{k+1} = (1 - \alpha)^k Q_1 + \sum_{i=1}^k \alpha (1 - \alpha)^{k-i} R_i.$$

Las recompensas más recientes reciben mayor peso. El peso de una recompensa antigua disminuye aproximadamente de forma exponencial:

$$\alpha (1 - \alpha)^{k-i}.$$

Por eso se denomina **promedio ponderado exponencialmente por recencia** (*exponential recency-weighted average*).

Si α es grande, por ejemplo $\alpha = 0.8$, la estimación cambia rápidamente ante nueva información: aprende y olvida rápido, pero puede resultar más ruidosa.

Si α es pequeño, por ejemplo $\alpha = 0.01$, la estimación cambia lentamente, es más estable y conserva información antigua durante más tiempo.

Si utilizamos una secuencia de tamaños de paso $\alpha_k(a)$, para garantizar convergencia suelen exigirse las condiciones

$$\sum_{k=1}^{\infty} \alpha_k(a) = \infty$$

y

$$\sum_{k=1}^{\infty} \alpha_k^2(a) < \infty.$$

La primera evita que el algoritmo deje de aprender demasiado pronto. La segunda hace que los cambios eventualmente sean suficientemente pequeños como para permitir la convergencia.

Por ejemplo,

$$\alpha_k = \frac{1}{k}$$

cumple ambas condiciones. En cambio, un valor constante

$$\alpha_k = \alpha$$

no cumple la segunda. Esto **no es necesariamente malo**: en un problema no estacionario queremos que $Q_t(a)$ siga cambiando para adaptarse a las modificaciones del entorno. Por eso los tamaños de paso constantes son muy útiles en RL.

Toda estimación necesita un punto de partida $Q_1(a)$. En vez de usar, por ejemplo, $Q_1(a) = 0$, podemos arrancar deliberadamente con valores demasiado altos, como

$$Q_1(a) = 5,$$

aunque las recompensas reales estén aproximadamente alrededor de cero. Esto se denomina **valores iniciales optimistas**.

Suponiendo que $Q_1(a) = 5$ para todas las acciones. Elegimos una acción y obtenemos $R = 1$; entonces, su estimación disminuye. Las demás todavía tienen $Q(a) = 5$, por lo que parecen mejores y el agente pasa a probarlas. El proceso se repite y el agente termina probando muchas acciones aun utilizando una política puramente *greedy*.

La ventaja es que se trata de una forma muy fácil de generar exploración y no requiere $\varepsilon > 0$.

El problema es que la exploración producida es **temporal**. Una vez que desaparece el optimismo inicial, el algoritmo deja de explorar. Por eso no funciona especialmente bien en problemas no estacionarios: si el entorno cambia después de mucho tiempo, el algoritmo no recibe un nuevo impulso para explorar.

El problema de ε -greedy es que, cuando explora, lo hace casi **a ciegas**: elige al azar entre las acciones. Eso significa que puede volver a probar acciones claramente malas y dejar otras, más prometedoras pero poco conocidas, en segundo plano. UCB (*Upper Confidence Bound*) intenta explorar con un poco más de criterio.

Fórmula UCB

Se elige

$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right].$$

La expresión tiene dos partes:

1. **Explotación:** $Q_t(a)$ favorece las acciones con mayor recompensa estimada.
2. **Exploración:** $c \sqrt{\frac{\ln t}{N_t(a)}}$ favorece las acciones sobre las que tenemos mayor incertidumbre.

$N_t(a)$ indica cuántas veces elegimos la acción. Si fue elegida pocas veces, $N_t(a)$ es pequeño y

$$\sqrt{\frac{\ln t}{N_t(a)}}$$

es grande. Por lo tanto, UCB la considera atractiva para explorar. Cuando la probamos muchas veces, $N_t(a)$ aumenta y el incentivo de exploración disminuye.

El parámetro $c > 0$ controla cuánto se valora la exploración:

- un c grande produce más exploración;
- un c pequeño produce más explotación.

La idea detrás de UCB

UCB puede interpretarse como la elección de la acción con el mayor **valor plausible optimista**:

$$\underbrace{Q_t(a)}_{\text{estimación}} + \underbrace{\text{incertidumbre}}_{\text{exploración}}.$$

Una acción puede ser elegida porque creemos que es buena o porque todavía existe una posibilidad razonable de que sea mucho mejor de lo que pensamos.

En el experimento del libro, UCB supera generalmente a ε -greedy. Sin embargo, se señala que resulta más difícil aplicar directamente estas ideas a problemas generales de RL, especialmente cuando existen:

- problemas no estacionarios;
- espacios de estados grandes;
- aproximación de funciones.

Hasta ahora, el agente siempre enfrentaba mas o menos **la misma situación**. Por eso, el problema era encontrar la mejor acción. En el aprendizaje por refuerzo completo existen diferentes situaciones, de modo que tenemos que aprender una relación

$$\text{situación} \longrightarrow \text{acción},$$

es decir, una **política**.

Supongamos que existen varios bandits diferentes y que cada vez que jugamos aparece uno distinto. Si no sabemos cuál apareció, el problema parece simplemente no estacionario. Pero imaginemos que recibimos una pista o **contexto**; por ejemplo:

- pantalla roja $\longrightarrow$ bandit A;
- pantalla verde $\longrightarrow$ bandit B.

Entonces podemos aprender, por ejemplo,

$$\text{rojo} \longrightarrow \text{acción 1}, \quad \text{verde} \longrightarrow \text{acción 2}.$$

Ya no buscamos una acción óptima global, sino una acción óptima **dependiendo del contexto**. Esto introduce el concepto de:

Política: situación $\longrightarrow$ acción

Los bandits contextuales están a mitad de camino entre los bandits simples y el aprendizaje por refuerzo completo. En los bandits contextuales:

- existe un estado o contexto;
- aprendemos qué acción tomar en cada contexto;
- la acción solamente afecta la recompensa inmediata.

En RL completo, además,

la acción también afecta el estado futuro.
--

Nos quedamos con que **aprender no es solamente elegir lo que hoy parece mejor; también hay que invertir algunas acciones en conseguir información para decidir mejor más adelante.**